Python Engineering - Class 10 Reference Guide

Advanced Functions

Reference guide to the syntax and tools used in this class. It explains what each element does, how it is written and when it is useful.

Quick map

Concept	Syntax	Idea
positional arguments	f(a,b)	Matched to parameters by position.
keyword arguments	f(a=1,b=2)	Matched by parameter name.
default argument	def f(g=9.81)	Used when the caller does not provide that argument.
*args	def f(*args)	Collect variable positional arguments into a tuple.
**kwargs	def f(**kwargs)	Collect variable keyword arguments into a dictionary.
unpack *	f(*sequence)	Expand a sequence into positional arguments.
unpack **	f(**mapping)	Expand a mapping into keyword arguments.
function object	operation = add	Reference to a function without calling it.
function call	add(2,3)	Execute function and obtain returned result.
higher-order function	apply_operation(add,5,3)	A function that receives another function.

1. positional arguments

Syntax: `f(a,b)`

Matched to parameters by position.

```
initial_energy(100.0, 50.0, 9.81)
```

Control question: before running the example, predict the result type and shape.

2. keyword arguments

Syntax: `f(a=1,b=2)`

Matched by parameter name.

```
initial_energy(h0=100.0, v0=50.0, g=9.81)
```

Control question: before running the example, predict the result type and shape.

3. default argument

Syntax: `def f(g=9.81)`

Used when the caller does not provide that argument.

```
initial_energy(100.0, 50.0)
```

Control question: before running the example, predict the result type and shape.

4. *args

Syntax: `def f(*args)`

Collect variable positional arguments into a tuple.

```
def average_measurements(*values):
    return sum(values) / len(values)
```

Control question: before running the example, predict the result type and shape.

5. **kwargs

Syntax: `def f(**kwargs)`

Collect variable keyword arguments into a dictionary.

```
def simulation_summary(**kwargs):
    print(kwargs)
```

Control question: before running the example, predict the result type and shape.

6. unpack *

Syntax: `f(*sequence)`

Expand a sequence into positional arguments.

```
measurements = [10.2, 10.5]
average_measurements(*measurements)
```

Control question: before running the example, predict the result type and shape.

7. unpack **

Syntax: `f(**mapping)`

Expand a mapping into keyword arguments.

```
config = {'h0':0.0,'v0':50.0,'g':9.81}
initial_energy(**config)
```

Control question: before running the example, predict the result type and shape.

8. function object

Syntax: `operation = add`

Reference to a function without calling it.

```
operation = add
```

Control question: before running the example, predict the result type and shape.

9. function call

Syntax: `add(2,3)`

Execute function and obtain returned result.

```
result = add(5, 3)
```

Control question: before running the example, predict the result type and shape.

10. higher-order function

Syntax: `apply_operation(add,5,3)`

A function that receives another function.

```
result = apply_operation(add, 5, 3)
```

Control question: before running the example, predict the result type and shape.

Common mistakes

- Confusing a function name with a function call using parentheses.
- Not checking types, indices or shapes before an operation.
- Using a tool because it works without being able to explain what it receives and returns.

Mental cheat sheet

```
positional arguments: f(a,b)
keyword arguments: f(a=1,b=2)
default argument: def f(g=9.81)
*args: def f(*args)
**kwargs: def f(**kwargs)
unpack *: f(*sequence)
unpack **: f(**mapping)
function object: operation = add
function call: add(2,3)
higher-order function: apply_operation(add,5,3)
```